

4/9/25

Task 9 - Implement exceptions and Exception handling in Python

Aims:

To implement Exceptions and Exceptional handling in Python.

9.1. Develop a python program that processes a list of student grades. The program is designed to allow the user to select a grade by specifying an index number. It also ensures that index is not out of range.

Algorithm:

1. Start the program
2. Initializes a list of grades (e.g., [85, 90, 78, 92, 88]).
3. Prompts the user to enter the index of the grade they wish to view.
4. Attempts to display the grade at the specified index.
5. If the index is out of range, catches the `IndexError` and prints an error message - "Invalid index. Please enter a valid index"

Program:

```
grades = [85, 90, 78, 92, 88]
print("Grades List:", grades)
```

try:

```
index = int(input("Enter the index of the grade they want to view:"))
print(f"The grade at index {index} is: {grades[index]}")
```

except `IndexError`:

```
print("Invalid index. Please enter a valid index.")
```

except `ValueError`:

```
print("Invalid input. Please enter a numerical index.")
```

Output:

Grades List: [85, 90, 78, 92, 88]

Enter the index of the grade you want to view

Invalid index. Please enter a valid index.

9.2. Develop a python calculator using python program which performs arithmetic operations by handling the exceptions. Try to perform by dividing the two numbers.

Algorithm:

1. Start the program
2. Prompts the user to enter two numbers : a numerator and a denominator
3. Attempts to divide the numerator by the denominator
4. If the denominator is zero, catches the ZeroDivision Error and displays an error message : "Error : Division by zero is not allowed."

Program:

```
def divide_numbers():
```

```
    try:
```

```
        numerator = float(input("Enter the numerator:"))
```

```
        denominator = float(input("Enter the denominator:"))
```

```
        result = numerator / denominator
```

```
        print(f"Result = {result}")
```

```
    except ZeroDivisionError:
```

```
        print("Error: Division by zero is not allowed.")
```

```
    except ValueError:
```

```
        print("Error: Please enter a valid number.")
```

```
divide_numbers()
```

Output:

Enter the numerator = 10

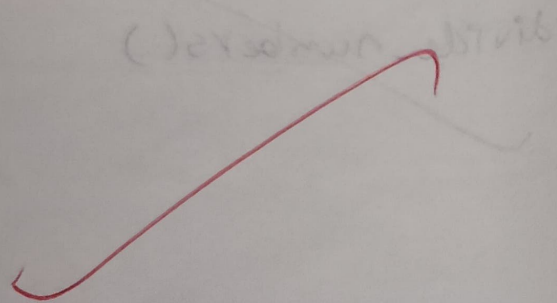
Enter the denominator = 0

ERROR!

Error: Division by zero is not allowed

Output:
Enter a number: 20

Eligible to Vote



9.3. You are building a python application to determine if a person is eligible to vote based on their age. According to the rules, only individuals who are 18 years or older are allowed to vote. To enforce this rule, you decide to create a custom exception called InvalidAgeException which will be raised whenever an age below 18 is entered.

Algorithm:

1. Define the custom exception
2. Prompt the user for input
3. Check if the age is below 18
4. Raise an exception if the condition is met
5. Handle the exception with a custom error message.

Program:

```
class InvalidAgeException(Exception):
    "Raised when the input value is less than 18"
    pass
number = 18
try:
    input_num = int(input("Enter a number:"))
    if input_num < number:
        raise InvalidAgeException
    else:
        print("Eligible to vote")
except InvalidAgeException:
    print("Exception - Invalid Age")
```

VEL TECH - CCE	
EX NO.	9
PERFORMANCE (5)	5
RESULT AND ANALYSIS (3)	3
VIVA VOCE (3) - Invalid Age	3
RECORD (4)	15
TOTAL (15)	
SIGN WITH DATE	

Result:

Thus the program for implement Exceptions and Exceptional handling is executed and verified